

Avaliação 2 - Otimização de Sistemas - Busca de mínimo global
para um função não convexa através de algoritmo da nuvem de
partículas e algoritmo genético

Pedro Yochinori Gushiken

11 de novembro de 2015

Algoritmo da Nuvem de Partículas

Retiramos as equações relevantes do trabalho:

$$r = 100 * (y - x^2)^2 + (1 - x)^2 \quad (1)$$

$$z = x * \sin(\sqrt{|x|}) - y * \sin(\sqrt{|y|}) \quad (2)$$

$$w_1 = r + z \quad (3)$$

$$w_3 = r - z \quad (4)$$

$$w_{4ant} = \sqrt{r^2 + z^2} \quad (5)$$

$$w_{13} = \sqrt{|w_1|} + \sqrt{|w_3|} - w_{4ant} * \cos x \quad (6)$$

$$\min(w_{13}) = ? \quad (7)$$

Para a minimização da função w_{13} definida para o trabalho, o algoritmo da nuvem de partículas com restrição no espaço de busca é de fácil implementação e tem capacidade de contornar o problema da não-convexidade da função w_{13} , que possui vários mínimos locais. O algoritmo da nuvem de partículas funciona através da movimentação dos elementos da nuvem dentro do espaço de busca de forma semi-aleatória. Determina-se, através de uma função de aptidão, escolhida aqui como a própria função w_{13} , o melhor valor de posição atingido por uma determinada partícula em relação a si mesma e em relação ao conjunto de todas as partículas. Após o registro destas variáveis, é feito o cálculo das velocidades de cada elemento usando constantes declaradas no código e números randômicos. Finalmente, com as velocidades calculadas, atualiza-se a posição da partícula para o passo seguinte.

Resultados

Declaramos um vetor de população com $m=101$ membros localizados em uma diagonal do espaço de busca por simplificação. A velocidade dos membros da população foi declarada como sendo inicialmente zero. O espaço de busca foi limitado a $-500 \leq x \leq 500$ e $-500 \leq y \leq 500$. As constantes c_1 e c_2 foram escolhida iguais a 2 e o peso w igual a 0.7.

Rodando o algoritmo com um número de iterações igual a 50, que é suficiente para garantir a convergência dos membros da população para dentro da bacia de mínimo, obtemos o valor da posição do mínimo global como (-122.02498, -425.82922), e o valor da função w_{13} para este ponto é -538.3587.

Podemos ter uma confiança maior que este é um mínimo local da função ao invocar a função w_{13} para valores de argumentos próximo à posição do mínimo encontrado, embora este teste não seja exaustivo.

Algoritmo Genético

Para a minimização da função w_{13} definida para o trabalho, o algoritmo da genético com restrição no espaço de busca tem a implementação um pouco mais difícil pois precisa manipular cadeias de bits, porém dispõe da mesma capacidade de contornar o problema da não-convexidade da função w_{13} , que possui vários mínimos locais, devido a seu caráter estocástico. O algoritmo genético funciona através do cruzamento e mutação dos elementos das cadeias de bits da população de forma semi-aleatória.

Determina-se, através de uma função de aptidão que deve ser sempre positiva, o valor de aptidão atingido por uma determinada cadeia de bits. Após o registro destas variáveis, é feito o cálculo da chance de cruzamento que cada cadeia dispõe em relação às outras, membros com aptidão maior tem mais chance de serem selecionados. Finalmente, uma pequena taxa de mutação é adicionada a cada bit de forma a garantir variedade populacional.

Resultados

Declaramos um vetor de população com $m=20$ cadeias de 16 bits randomizadas. A aptidão foi escolhida como $f(x) = 1/(w_{13}+600)$, que é positiva dentro de todo o espaço de busca e máxima no ponto de mínimo global da função. O espaço de busca foi limitado a $-500 \leq x \leq 500$ e $-500 \leq y \leq 500$. As constantes pertinentes a mutação e crossover foram escolhidas iguais a 0.01 e 0.75 respectivamente.

Rodando o algoritmo com um número de iterações igual a 36, que é suficiente para garantir a convergência dos membros da população para dentro da bacia de mínimo, obtemos o valor da posição do mínimo global como $(-132.71815, -425.27714)$, e o valor da função w_{13} para este ponto é -529.23716.

Podemos ter uma confiança maior que este é um mínimo local da função ao invocar a função w_{13} para valores de argumentos próximo à posição do mínimo encontrado, embora este teste não seja exaustivo e não exista garantia de que o ponto é realmente o mínimo da bacia encontrada. De fato, o algoritmo da nuvem de partículas encontra um valor menor para w_{13} , embora deva ser notado que a população declarada para a nuvem de partículas é muito maior e o algoritmo rodou durante mais iterações. Apesar dos resultados convergirem não conseguimos cumprir o objetivo expresso do trabalho de comparar os dois algoritmos de forma justa, pois seria necessário afinar os parâmetros de cada um para que consumissem aproximadamente o mesmo custo computacional.

Anexo

Função de Plotagem

```
clear
//
[x,y]=meshgrid(-500:5:500,-500:5:500);
//
z=-x.*sin(sqrt(abs(x)))-y.*sin(sqrt(abs(y)));
x=x/250;
y=y/250;
// r: Rosenbrock's function
r=100*(y-x.^2).^2+(1-x).^2;
r1=(y-x.^2).^2+(1-x).^2;
rd=1+r1;
//
x1=25*x;
x2=25*y;
xs =-10:0.1:10;
ys =-10:0.1:10;
a=500;
b=0.1;
c=0.5*%pi;
//
F10=-a*exp(-b*sqrt((x1.^2+x2.^2)/2))-exp((cos(c*x1)+cos(c*x2))/2)+exp(1);
//
[n nx]=size(xs);
[n ny]=size(ys);
for i=1:nx
for j=1:ny
zsh(i,j)=0.5-((sin(sqrt(xs(i)^2+ys(j)^2)))^2-0.5)./(1+0.1*(xs(i)^2+ys(j)^2))^2;
end
end
//
Fobj=F10.*zsh//+a*cos(x1/30);
//
//surf(x1,x2,Fobj)
//
w=r.*z;
w1=r+z;
w2=z-r1;
w3=r-z;
```

```

w4ant=sqrt(r.^2+z.^2);
w4=sqrt(r.^2+z.^2)+Fobj;
w5=w-0.5*w1;
w6=w+w2;
w7=w1+w4;
w8=w1-w4;
w9=w2-w4;
w10=w2+w4;
w11=w3-w4;
w12=r+w4.*cos(y);
w13=sqrt(w1)+sqrt(w3)-w4ant.*cos(x);
w14=z.*exp(sin(r1));
w15=z.*exp(cos(r1));
w16=w14+w4;
w17=-w14+w4;
w18=-w15+w4;
w19=exp(-r1).*z;
w20=x.*z;
w21=(x+y).*z;
w22=(x-y).*z;
w23=z./rd;
w24=(x-y).*w23;
w25=(x+y).*w23;
w26=-w4./rd;
w27=w4+w23;
w28=w4-w23;
w29=w14+w23;
w30=w4+w14+w23;
w31=w21+w22;
w32=w21+w23;
w33=w22+w25;
w34=w22+w26;
w35=w23+w27;
w36=w23+w28;
w37=w23+w29;
w38=w23+w30;
w39=w25+w30;
w40=w27+w30;
x = x*250;
y = y *250;
//
surf(x,y,w4);

```

Código usado para a solução

Declaração das funções

```

//definição da func. rosenbrock segundo o arquivo enviado
function [x] = funcR(a,b)
x = 100*(0.004*b-(0.004*a)^2)^2+(1-0.004*a)^2

```

```

endfunction

//definição da função z
function [x] = funcZ(a,b)
x = a*sin(sqrt(abs(a))) - b*sin(sqrt(abs((b))))
endfunction

//definição das funções pertinentes à w13
function [x] = funcw1(a, b)
a1 = funcR(a, b)
a2 = funcZ(a, b)
x = a1+a2
endfunction

function [x] = funcw3(a, b)
a1 = funcR(a,b)
a2 = funcZ(a,b)
x = a1-a2
endfunction

function [x] = funcw4ant(a,b)
a1 = funcR(a,b)
a2 = funcZ(a,b)
x = sqrt(a1^2 + a2^2)
endfunction

//definição de w13, func. a ser minimizada
function [x] = funcw13(a,b)
x = sqrt(abs(funcw1(a,b))) + sqrt(abs(funcw3(a,b))) - funcw4ant(a,b)*cos(0.004*a)
endfunction

```

Algoritmo da Nuvem de Partículas

```

//inicialização do vetor de população de tamanho m=101, localizados numa diagonal do espaço de
xpop = [-500:10:500;-500:10:500]
//inicialização do vetor de melhor posição alcançada para um membro da pop.
ppop = [-500:10:500;-500:10:500]
//inicialização do vetor velocidade para cada membro da população
vpop = zeros(2, 101)
//inicialização das variáveis de melhor/mínimo global, g1 corresponde ao eixo x e g2 ao eixo y
g1 = 0
g2 = 0
//constantes pertinentes ao método usado (particle swarm, enxame de partículas)
c1 = 2
c2 = 2
peso = 0.70

for k=1:50

```

```

// condição de término é a de rodar 100 iterações do código
for l=1:101
//para cada um dos membros da população definida rode:
    if funcw13(xpop(1,l),xpop(2,l))<funcw13(ppop(1,l),ppop(2,l)) then
        ppop(1,l) = xpop(1,l)
        ppop(2,l) = xpop(2,l)
        //se o valor de w13 é menor para esta posição xpop que ppop, grave em ppop
    end
    if funcw13(xpop(1,l),xpop(2,l))<funcw13(g1,g2) then
        g1 = xpop(1,l)
        g2 = xpop(2,l)
        //se o valor de w13 é menor para esta posição de xpop que g1 e g2, grave em g1 e g2
    end
    r11 = rand()
    r12 = rand()
    r21 = rand()
    r22 = rand()
//escolha dos números randômicos a cada iteração do laço
    vpop(1,l) = peso*vpop(1,l) + c1*r11*(ppop(1,l) - xpop(1,l)) + c2*r12*(g1 - xpop(1,l))
    vpop(2,l) = peso*vpop(2,l) + c1*r21*(ppop(2,l) - xpop(2,l)) + c2*r22*(g2 - xpop(2,l))
//determinação das velocidades
    xpop(1,l) = xpop(1,l) + vpop(1,l)
    xpop(2,l) = xpop(2,l) + vpop(2,l)
//determinação das posições
    if xpop(1,l)>=500 then
        xpop(1,l) = 500
    end
    if xpop(1,l)<=-500 then
        xpop(1,l) = -500
    end
    if xpop(2,l)>=500 then
        xpop(2,l) = 500
    end
    if xpop(2,l)<=-500 then
        xpop(2,l) = -500
    end
//projeção do espaço de busca nos eixos x e y (nos limitamos ao retângulo delimitado por -500<=x<=500 e -500<=y<=500)
end
end

```

Algoritmo Genético e funções pertinentes à cadeias de bit

```

//retornar a cadeia de bits para o membro da população index em xpop ou ypop
function [x] = extCBit(a, index)
    s = size(a)
    x = zeros(s(1), 1)
    for h=1:s(1)
        x(h, 1) = a(h, index)
    end
end

```

```

endfunction

// converter o valor da cadeia de bits para o valor real dados os limites
function [x] = retCBit(a, LimS, LimI)
    x = 0
    for h=1:length(a)
        x = x + a(h)*2^(h-1)
    end
    x = LimI+x*(LimS - LimI)/(2^(length(a)+1)-1)
endfunction

//função de randomização de um bit
function [x] = ranBit(bit)
x = bit
    if rand()>0.5 then
        x= 1
    else
        x=0
    end
endfunction

//função de randomização de uma cadeia de bits
function [a] = ranCBit(a)
    for h=1:length(a)
        a(h)= ranBit(a(h))
    end
endfunction

//cálculo da aptidão de duas cadeias de bit x e y
function [x] = aptCBit(a, b)
    a1 = retCBit(a, 500, -500)
    a2 = retCBit(b, 500, -500)
    x = 1/(funcw13(a1, a2) + 600)
endfunction

//função crossover entre duas cadeias de bit
function [x, y] = crossover(a,b)
    l = rand()
    pcorte = (1 - modulo(l, 1/length(a)))*length(a)
    a1 = a
    a2 = b
    for h=1:pcorte
        a(h) = a2(h)
        b(h) = a1(h)
    end
    x = a
    y = b
endfunction

//função mutação para cada bit

```

```

function [x] = mutacao(a,mut)
    x=a
    if rand()<mut then
        if a==0 then
            x=1
        else
            x=0
        end
    end
end
endfunction

function [x] = mutCBit(a)
    a1 = length(a)
    x = zeros(a1, 1)
    for h=1:length(a)
        x(h, 1) = mutacao(a(h, 1), mut)
    end
end
endfunction

```

```

// N é o numero de bits nas cadeias de bits, 10 bits para x e 10 bits para y
s = size(xpop)
m = s(1)
N = s(2)
//variáveis auxiliares
cbitANx = zeros(16, 1)
cbitANY = zeros(16, 1)
ubitANx = zeros(16, 1)
ubitANY = zeros(16, 1)
//inicialização melhor par ordenado de cadeias
mbitAPTx=zeros(16,1)
mbitAPTy=zeros(16,1)
//aptidão total
VarAPTt = 0
//chance de seleção para cada membro da população
p = zeros(1,20)
mut = 0.01
cros = 0.75
//inicialização dos membros da população
xpop = zeros(16,20)
ypop = zeros(16,20)
for h=1:16
    for e=1:20
        xpop(h,e) = ranBit(xpop(h,e))
        ypop(h,e) = ranBit(ypop(h,e))
    end
end
end

```

```

//início do algoritmo
// k é o número de iterações do algoritmo
for k=1:36
    for g=1:m
        //cálculo do total de todas as aptidões
        cbitANx = extCBit(xpop, g)
        cbitANY = extCBit(ypop, g)
        VarAPTt = VarAPTt + aptCBit(cbitANx, cbitANY)
        if aptCBit(cbitANx, cbitANY)>aptCBit(mbitAPTx, mbitAPTy) then
            mbitAPTx = cbitANx
            mbitAPTy = cbitANY
        end
    end
    //cálculo das probabilidades de seleção de cada membro
    for g=1:m
        p(1, g) = aptCBit(cbitANx, cbitANY)/VarAPTt
    end

    for g=1:m
        if rand()<cros then
            for u=1:m
                if rand()<p(1, u) then
                    ubitANx = extCBit(xpop, u)
                    ubitANY = extCBit(ypop, u)
                    [ubitANx, cbitANx] = crossover(ubitANx, cbitANx)
                    [ubitANY, cbitANY] = crossover(ubitANY, cbitANY)
                    for h=1:N
                        xpop(g, h) = cbitANx(g, 1)
                        ypop(g, h) = cbitANY(g, 1)
                        xpop(u, h) = ubitANx(u, 1)
                        ypop(u, h) = ubitANY(u, 1)
                    end
                end
            end
        end
    end
    //mutação das cadeias de bit em x e y para o membro g da população
    for g=1:m
        cbitANx = extCBit(xpop, g)
        cbitANY = extCBit(ypop, g)
        cbitANx = mutCBit(cbitANx)
        cbitANY = mutCBit(cbitANY)
        for h=1:N
            xpop(g, h) = cbitANx(g, 1)
            ypop(g, h) = cbitANY(g, 1)
        end
    end
end
end

```